Suggesting General Ledger Codes for Department for Transport Spending: An Applied Machine Learning Study

Patrick Gilliland

Udacity

AI Mastery Capstone: Applied Machine Learning

Udacity mentor

October 11, 2026

Suggesting General Ledger Codes for Department for Transport Spending: An Applied Machine Learning Study

Overview

In this Machine Learning Analysis Report I describe a model that suggests the Expense Type, the general ledger code, for each Department for Transport (DfT) spend line over £25,000. In accounting terms this is general ledger account classification, the assignment of an account code to each posting line (Beļskis et al., 2021), and it is a supervised learning (classification) task with many classes. The dataset is "Spend over £25,000 in the Department for Transport and its arms-length bodies", listed by DfT on data.gov.uk at https://www.data.gov.uk/dataset/804be0c6-ae01-4899-bbb9-2317dff1a4f9/financial-transactions-data-dft (Department for Transport, 2026). The chosen model is a logistic regression built with scikit-learn (Pedregosa et al., 2011), with a reject option that leaves the lines it is unsure of to a person.

Dataset Description

The dataset is DfT's monthly list of payments over £25,000 by the department and its arm's length bodies, one row per ledger line, and none of my earlier capstone projects, AI Programming Foundations and Statistical Analysis, used it. A script stacks the 48 monthly files from April 2022 to March 2026, changing no value, into a file of 126,346 rows and 13 columns, 10 from DfT and 3 recording each row's source, well above my assumed floor of 500 rows and 6 columns.

After I decided how to treat each anomaly I found, 125,360 lines are used for modelling. The target is the Expense Type: the model predicts the 50 commonest types in the months it is fitted on, which I call the kept types, or "Other" for every other type. The key features are the

item text, the supplier, the Expense Area, the paying body and the amount. The data is public sector information licensed under the Open Government Licence v3.0 (Department for Transport, 2026).

Modeling Approach

Data Preparation

I listed 31 kinds of anomaly and decided how to treat each. No row leaves the data: three kinds are kept out of modelling, the 657 blank rows, the 329 extra copies of exact duplicates and the unreadable amounts, of which there are none, and the other 28 kinds are kept, 10 of them flagged. I read the 15,989 missing item texts as no text, so those lines are coded from their other inputs, and fixed 29 dates written with dots or spaces by rule, to give the column one format. The date is not an input, as the model always codes months it has never seen.

I merged label spellings by fixed rules, and a label cut short at 20 characters into its full label only where the training months confirmed it, taking 1,054 labels to 856. The spellings are chosen on the training months only, because preprocessing may use only training data (Kapoor et al., 2024).

The model reads five inputs from each line:

- The item text, as term frequency-inverse document frequency (TF-IDF) weights on words and word pairs (scikit-learn developers, 2026b).
- The supplier, as TF-IDF weights on runs of 2 to 4 characters, which resist misspellings, so a name written with "Ltd" and with "Limited" shares most of its features (scikit-learn developers, 2026b).

- The Expense Area, the part of the body that spent the money, often its cost centre, one-hot encoded with one yes-or-no column per value on at least 5 training lines (scikit-learn developers, 2026e).
- The paying body, encoded the same way.
- The amount, logged and standardised, as amounts run from £25,000.00 to £786,575,788.00 and many estimators behave badly on very different scales (scikit-learn developers, 2026e).

These steps sit in one scikit-learn pipeline, fitted on the training months only, which keeps the test data out of every fitted step (scikit-learn developers, 2026a). I did not resample the types, because resampling distorts the predicted probabilities (van den Goorbergh et al., 2022), and the reject option depends on them.

The Time-Ordered Hold-Out

I split the lines by month, not at random: training runs to June 2025, validation from July to September 2025, and test from October 2025 to March 2026. In use the model always codes months after the ones it learned from, and a random split would cause temporal leakage (Kapoor & Narayanan, 2023).

I made every choice on validation. Of the windows I compared, from the latest 12 months to all 39, each with its own kept types, the latest 12 months scored best, though only by about half a percentage point. So I fitted the model on July 2024 to June 2025: 31,462 lines, 81.05% of them in kept types.

The models scored the test months once, at the end, because choosing on the data a result is reported on biases it (Cawley & Talbot, 2010). The anomaly list counted every month, but no choice used the test months.

Why Logistic Regression

On validation, logistic regression's first choice was right for 92.0% of kept-type lines, and scikit-learn's HistGradientBoosting, a gradient-boosted tree model, got 89.7%. The rule, fixed in advance, was that the higher score wins unless the two are within half a percentage point, when the simpler logistic regression wins. Logistic regression won on the score alone.

Tree ensembles usually lead on medium-sized tables (Grinsztajn et al., 2022), but my reading is that text changes the picture here. Most of the 15,635 features are supplier character runs and item-text words, which suit fast linear models (scikit-learn developers, 2026b). The boosting model needs dense input, so each text block is compressed to 100 columns.

The boosting model trained properly, with small regularised trees and a lower learning rate than the default: its training loss fell from 2.93 to 0.079 over 115 iterations, with early stopping on a random tenth of the training lines, not a time-ordered check. Logistic regression still beat this boosting pipeline as I set it up. Sparse text and categories did not need a neural network, so I did not use PyTorch.

Why These Evaluation Metrics

The metrics fit the model type, the task and the dataset. The model is a probabilistic classifier, so its confidence can be measured and used, as the last two metrics do. The task is multi-class account classification, where in my judgement a wrong code costs more than a line left for a person. The dataset has many unequal types, and it changes over time.

- **Accuracy on kept-type lines**, the primary metric, fixed before the test months were read, is the share of kept-type lines whose first-choice type is exactly right. It shows how well the model ranks types, so coverage and the accuracy on the lines the model codes go with it, both defined below.

- **Macro-averaged F1** (macro F1) weights each kept type equally, so rare types count (scikit-learn developers, 2026d). F1 balances how often a predicted type is right and how often a true type is found.
- **Top-3 accuracy** asks whether the right type is among the three highest-ranked suggestions (scikit-learn developers, 2026d), as a reviewer picks from a list.
- **Coverage** is the share of lines the model codes by itself. The reject option lets the model abstain and defer a line to a person (Hendrickx et al., 2024), trading coverage for accuracy (El-Yaniv & Wiener, 2010), as the risk-coverage curve shows (Geifman & El-Yaniv, 2017).
- **Expected calibration error** (ECE), over 15 equal-width bins, measures the gap between confidence and accuracy (Guo et al., 2017). The threshold needs only a confidence that ranks right lines above wrong ones, but a reviewer who trusts it needs it to mean what it says.

Assumptions the Model Makes

- **The log-odds are linear.** scikit-learn's logistic regression is a linear model of the log-odds of each type, regularised by default (scikit-learn developers, 2026c). I set its regularisation setting on validation, at C = 10.
- **The lines are independent.** This holds only roughly: one transaction can carry several lines, and some transaction numbers recur across the sets.
- **The cost centre is known at coding time**, from the purchase order or the budget holder's approval. That is my inference from common enterprise resource planning (ERP) practice, so I also report the model without it.

- **The future looks like the training months**, which concept drift breaks (Gama et al., 2014), as the test months show.

Results

Table 1 sets out the key metrics. The supplier-lookup baseline gives each supplier its commonest training type.

Table 1

Key Metrics for the Chosen Model, the Gradient-Boosted Tree Model and the Supplier-Lookup Baseline

Metric	Chosen model, validation	Chosen model, test	Boosted trees, test	Baseline, test
Accuracy on kept-type lines	.920	.884	.867	.586
Macro F1, 49 kept types	.833	.782	.769	.515
Top-3 accuracy on kept-type lines	.990	.985	.979	
"Other" lines given a real type	.182	.448	.431	.470
ECE, 15 bins	.028	.135	.156	
Coverage at the threshold of .86	.637	.613		
Accuracy on the lines the model codes	.951	.774		

Note. Values are shares from 0 to 1. The first five rows score the first choice, before the reject option, and the last two apply the threshold of .86. A blank cell means the metric does not apply: the baseline gives no probabilities, and the reject option applies to the chosen model only. The test months hold 16,283 lines, 10,764 with a kept type.

The Chosen Model Against the Baselines

On the test months the chosen model's first choice was right for 88.4% of kept-type lines, down from 92.0% on validation. The supplier-lookup baseline got 58.6%, a gap of 29.8 percentage points between those figures, and the always-"Other" baseline, which codes nothing and so scores 0, is left out of Table 1.

Macro F1 was .782 over the 49 kept types present, so performance is uneven across types. For kept-type lines, top-3 accuracy was 98.5%.

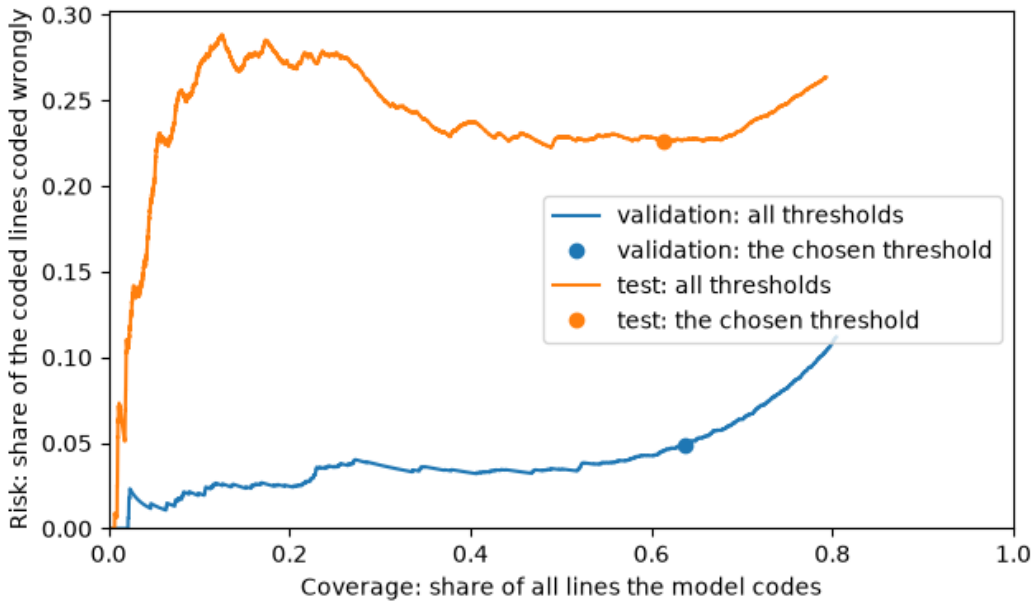
The weak spot is "Other": the first choice was a real type for 44.8% of the test "Other" lines, against 18.2% on validation. Without Expense Area the model was weaker, at 85.6% on kept-type lines, so the cost centre helps, on the assumption above.

Confidence, Coverage and Results per Body

I set the threshold of .86 on validation as the lowest that met my target of 95% accuracy on the lines the model codes, giving coverage of 63.7% with 95.1% accuracy on those lines (Figure 1). On the test months it coded 61.3% of lines with 77.4% accuracy on them, below the 88.4% above because it counts every line coded, "Other" lines included, and every "Other" line it codes is wrong.

Figure 1

Risk and Coverage of the Chosen Model at Every Threshold

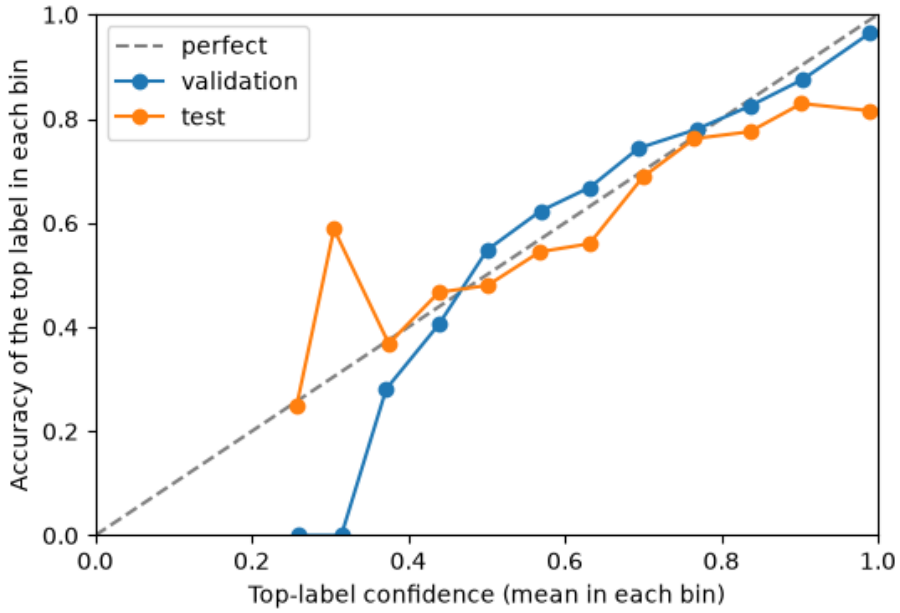


Note. Data from Department for Transport (2026). Risk is the share of the lines the model codes that it codes wrongly, and coverage is the share of all lines it codes. Each curve traces every threshold, and each dot marks the chosen threshold of .86.

Figure 1 shows the test curve above the validation curve at almost every coverage, so no threshold set on validation removes the confident mistakes. Figure 2 shows why: the ECE was .028 on validation and .135 on test, where lines the model was 99.0% sure of were right 81.5% of the time.

Figure 2

Calibration of the Chosen Model on Validation and Test

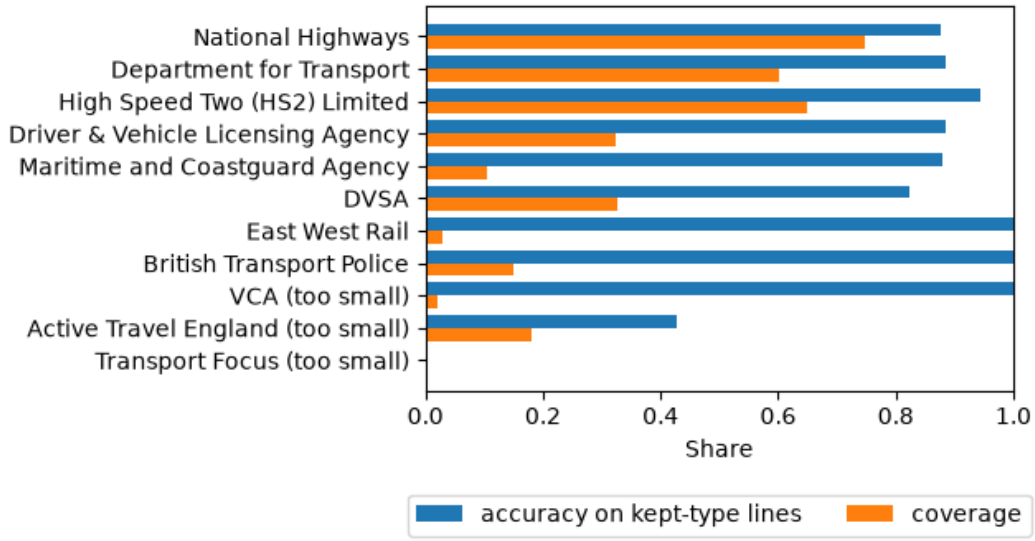


Note. Data from Department for Transport (2026). Each point is one of the 15 bins of the model's top-label confidence. A point below the dashed line marks a model more confident than it is right.

Results for each DfT body are a disaggregated evaluation (Mitchell et al., 2019). Figure 3 shows accuracy on kept-type lines high for every body large enough to judge, though East West Rail's and British Transport Police's 1.00 rest on only 13 and 48 kept-type lines, and coverage varying widely. Three bodies have fewer than 200 lines in all, my assumed floor for judging a body.

Figure 3

Accuracy and Coverage for Each Body in the Test Months



Note. Data from Department for Transport (2026). The two bars measure different lines: accuracy on kept-type lines scores the first choice on a body's kept-type lines, before the threshold, and coverage is the share of all its lines the model codes at the threshold of .86. "Too small" marks a body under 200 test lines, my assumed floor, and Transport Focus has no kept-type lines. DVSA is the Driver and Vehicle Standards Agency and VCA the Vehicle Certification Agency.

The core department and High Speed Two (HS2) Limited get 96.4% and 98.8% accuracy on the lines the model codes, at coverage of 60.1% and 64.9%. The model codes only 32.4% of the Driver and Vehicle Licensing Agency's (DVLA) lines and 32.6% of the Driver and Vehicle Standards Agency's (DVSA). It codes a smaller share still of three others' lines, 14.9% of British Transport Police's, 10.4% of the Maritime and Coastguard Agency's and 3.0% of East West Rail's, as most of these five bodies' lines fall outside the kept types. National Highways gets 66.4% at coverage of 74.8%, because at the threshold the model codes 80.8% of its "Other" lines.

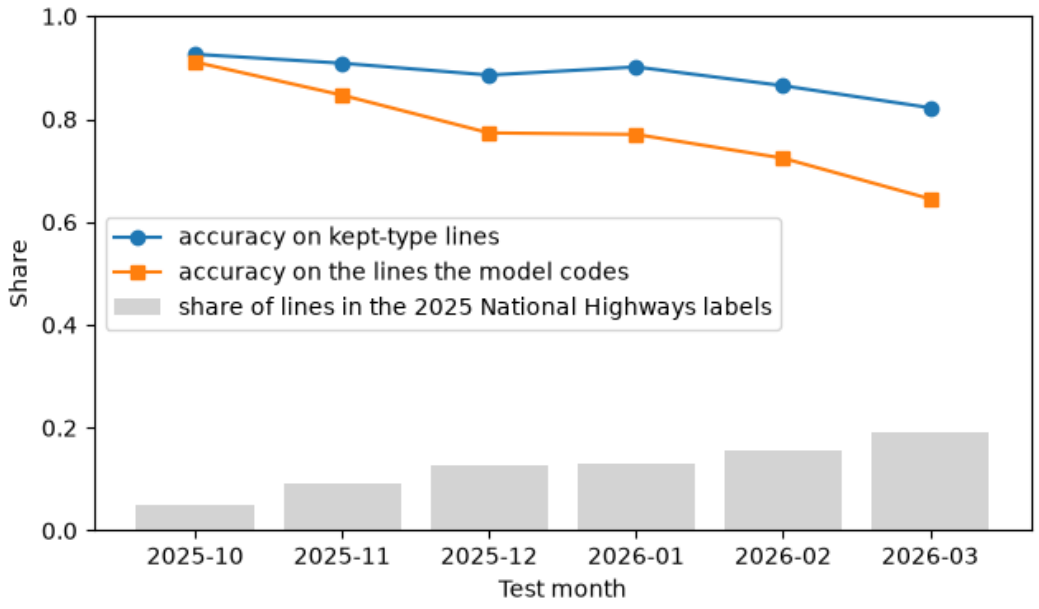
Concept Drift in the Test Months

From April 2025 National Highways began using new capital labels, 15 in all, which I call the 2025 National Highways labels. By June 2025, 12 had only 34 training lines between them, too few to be kept types, and 3 were first seen after training. The test months hold 2,091 of their lines, 12.8% of all test lines, all truly "Other". The model coded 2,039 of them, all wrongly, mostly to older capital accounts. These 2,039 lines are 96.7% of the test "Other" lines it coded at the threshold.

Figure 4 shows their share growing from 5.1% of lines in October 2025 to 19.3% in March 2026, while the accuracy on the lines the model codes falls from 91.1% to 64.5%, at coverage of 64.1% and 59.1%. Accuracy on kept-type lines also fell, from 92.6% to 82.2%, which these labels cannot cause, so the rest of the ledger drifts too, more slowly.

Figure 4

Monthly Accuracy as the 2025 National Highways Labels Grow



Note. Data from Department for Transport (2026). The grey bars are the share of each month's lines in the 2025 National Highways labels.

I read this as concept drift (real drift), a change in the target for the same inputs (Gama et al., 2014): as far as the data show, the same kind of spend moved to new codes. It also brought novel classes, as drift can (Hendrickx et al., 2024).

As a diagnostic only, chosen after seeing the test months, I left out these labels. The accuracy on the lines the model codes is then 97.2%, at coverage of 55.9%, against the headline 77.4% at coverage of 61.3%. So the label change, where the model is confident and wrong, is its largest failure.

Interpretation for a Non-Technical Audience

Does it work? Yes, as a first release that suggests the code for each line for a person to approve.

For lines whose code is one of the 50 codes the model learned, its first suggestion on the six test months was right 88.4% of the time. A simple rule giving each line its supplier's usual code got 58.6%, so a standard model reading five fields of each line beat the rule by 29.8 percentage points.

The model also says how sure it is, and codes a line by itself only when very sure. It coded 61.3% of all lines, and 77.4% of those were right. That is lower than 88.4% because it includes lines whose code the model had hardly seen.

Most of the wrong ones come from one change: National Highways' new capital codes from April 2025, where the model gave 2,039 lines the older codes. With the 2025 National Highways labels set aside, as a diagnostic check and not the result, it coded 55.9% of the rest, and 97.2% of those were right.

This first pass sets the scope of the real work, closing three gaps:

- **The label change.** A mapping from the new codes to the old, agreed with the finance team and tested, and a monthly check for the next change.
- **The remaining errors.** Even outside the label change a few in every hundred coded lines are wrong: 97.2% were right in the diagnostic, at coverage of 55.9%, and 95.1% on validation, at coverage of 63.7%. They can be studied, then fixed or always sent to a person.
- **The lines it does not code.** About four in ten go to a person, which more kept types could reduce.

Why not tune it to 100%? A higher threshold does not remove a mistake the model is sure of, so in the first release a person approves every line before it goes into the accounts.

Each approval or correction becomes a labelled example for the next round of training, so the approach should pay back, though a pilot must show it. The next step, if the savings justify it, is to map the changed accounts, review the remaining errors and measure again.

Limitations and Potential Bias

A Limitation and a Tradeoff: Coverage Against Accuracy

The reject option is a tradeoff, set by the threshold. On validation, a threshold of .50 codes 76.9% of lines with 90.9% accuracy on the lines the model codes, .86 codes 63.7% with 95.1%, and .90 codes 61.1% with 95.5%. A higher threshold raises that accuracy and sends more lines to a person.

The tradeoff holds only while the data stays as it was: "if a model updates on new data, the old manually set threshold may be invalid" (Sculley et al., 2015, p. 7).

The model's own limitation is that it cannot predict a code it has hardly seen, so the drift hit it with no warning from its confidence. Ovadia et al. (2019) found that "while calibrating on the validation set leads to well-calibrated predictions on the test set, it does not guarantee calibration on shifted data" (p. 5). The impact was the fall from 95.1% to 77.4% accuracy on the lines the model codes, at coverage of 63.7% and 61.3%.

Dataset Limitations

- **One department.** The results describe DfT's ledger only.
- **Labels cut short and merged.** 22,261 lines carry a label of exactly 20 characters. Of 154 pairs of a cut-short label and a possible full label, the cautious rules keep 131 apart, and their merges are evidence, not proof, so the target is partly built by them.
- **Redacted suppliers.** On 3,563 lines the supplier input holds the word "REDACTED", though most are the subsidy lines in the first finding below, which predate the training window.
- **The cost centre.** If it is not known at coding time, the results without Expense Area apply.

I also made three findings about the published data, set against HM Treasury's guidance for publishing spend over £25,000 (HM Treasury, 2017):

- The core department redacted the supplier on 3,476 of its 7,121 private-sector subsidy lines, until April 2024.
- 56 lines go to suppliers named only as a placeholder, such as a "one time" vendor.
- 3,256 lines, 33.9% of published value, go from one DfT body to another, found by matching supplier names. The guidance asks for them to be published. Where the

receiver also publishes its spending, a simple sum counts the money twice, at most 9.8% of the total, an effect on budget totals outside the scope of this work.

These are findings about the data, not claims of wrongdoing, and none feeds the model.

Sources of Uncertainty

- **One test period.** The results come from one six-month hold-out, so they carry uncertainty one period cannot measure, and the monthly fall may continue.
- **Reproducibility.** The notebook asks the maths library for 16 threads, because the thread count sets the order of floating-point sums, and a machine with fewer cores may run fewer. The search chooses between close scores: C = 10 and C = 30 scored .920 and .918 on validation, inside the half-point margin I set for the model choice. So on such a machine the search can pick a neighbouring C, and so a different threshold, and a rerun may not match the printed figures exactly.

Bias and Responsible Use

A potential source of bias is how well each body is served. One kept-types list serves every body, so a body whose lines mostly fall outside the kept types sends most of them to a person, as the results per body show. The bodies are organisations, not people, so in my view disaggregated evaluation (Mitchell et al., 2019) carries over and fairness measures built for people do not.

The step taken to reduce that risk is the reject option, which sends a line the model is unsure of to a person. It misses confident mistakes, as National Highways' 66.4% accuracy on the lines the model codes, at coverage of 74.8%, shows.

The further step I propose is suggest-and-review: a person approves every line, and a body or account that changes goes to a person until mapped. For a kept-type line the right code

was among the first three suggestions 98.5% of the time, and where the right code is not among them, the reviewer codes the line from scratch.

It has two risks. Automation bias "cannot be prevented by training or instructions" (Parasuraman & Manzey, 2010, p. 381), and a model retrained on reviewed suggestions partly learns its own output, a feedback loop (Sculley et al., 2015). The guard is a second check on a sample of approvals, and a small random share coded without the suggestion.

Whether a remaining error matters is a question of materiality: International Standard on Auditing (UK) 320 notes that frameworks treat a misstatement as material if it could "reasonably be expected to influence the economic decisions of users" (Financial Reporting Council, 2025, para. 2). Suggest-and-review is informed by that and by government guidance on artificial intelligence (AI), which asks that humans "validate any high-risk decisions influenced by AI" (Government Digital Service, 2025, Principle 4).

Potential Improvements

Each improvement answers a limitation named above, and the last is also a next step:

- **A mapping for changed accounts**, agreed with the finance team and tested, should remove most of the largest source of error. It answers the model's own limitation, that it cannot predict a code it has hardly seen.
- **Monthly monitoring** of coverage and the accuracy on the lines the model codes, against the reviewed lines, with the threshold reset when they slip (Sculley et al., 2015). It answers the limit on the tradeoff, which holds only while the data stays as it was.
- **Controls, not tuning**, are where near-100% accuracy on posted lines would come from: approval of each line, a second check on a sample, and changed accounts sent to a person. Even good control gives "reasonable but not absolute assurance" (Committee of

Sponsoring Organizations of the Treadway Commission, 2013, p. 9). They answer the limit of the reject option, that it misses confident mistakes.

- **Retraining on corrected postings**, so the model learns a new code once enough of its lines are reviewed. It also answers the model's own limitation, a code it has hardly seen.
- **A per-body gate and more kept types**, to send a changing body's lines to a person and raise coverage for the small bodies. They answer the potential source of bias, how well each body is served.
- **A stated risk target** before any automatic posting: a threshold can hold a chosen error rate with high probability on data like the data it was set on (Geifman & El-Yaniv, 2017), so it needs checking on later months. It answers the uncertainty of one test period.
- **A neural network for tabular data** (Gorishniy et al., 2021), built in PyTorch once corrected postings grow the data, could take the item text as one more input. It answers a limit of the chosen model, its assumption that the log-odds are linear.

References

Beļskis, Z., Zirne, M., Slaidiņš, V., & Pinnis, M. (2021). Natural language based posting account classification. *Baltic Journal of Modern Computing, 9*(2), 180-194. https://doi.org/10.22364/bjmc.2021.9.2.03

Cawley, G. C., & Talbot, N. L. C. (2010). On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research, 11*, 2079-2107. https://jmlr.org/papers/v11/cawley10a.html

Committee of Sponsoring Organizations of the Treadway Commission. (2013, May). *Internal control: Integrated framework: Executive summary*. https://www.coso.org/_files/ugd/3059fc_1df7d5dd38074006bce8fdf621a942cf.pdf

Department for Transport. (2026). *Spend over £25,000 in the Department for Transport and its arms-length bodies* [Data set]. data.gov.uk. Retrieved October 8, 2026, from https://www.data.gov.uk/dataset/804be0c6-ae01-4899-bbb9-2317dff1a4f9/financial-transactions-data-dft

El-Yaniv, R., & Wiener, Y. (2010). On the foundations of noise-free selective classification. *Journal of Machine Learning Research, 11*, 1605-1641. https://jmlr.org/papers/v11/el-yaniv10a.html

Financial Reporting Council. (2025). *International Standard on Auditing (UK) 320 (Revised June 2016, updated September 2025): Materiality in planning and performing an audit*. https://www.frc.org.uk/documents/8755/ISA_UK_320_Revised_June_2016_Updated_September_2025.pdf

Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys, 46*(4), Article 44. https://doi.org/10.1145/2523813

Geifman, Y., & El-Yaniv, R. (2017). Selective classification for deep neural networks. *Advances in Neural Information Processing Systems, 30*. https://proceedings.neurips.cc/paper_files/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cf1-Abstract.html

Gorishniy, Y., Rubachev, I., Khrulkov, V., & Babenko, A. (2021). Revisiting deep learning models for tabular data. *Advances in Neural Information Processing Systems, 34*, 18932-18943. https://proceedings.neurips.cc/paper_files/paper/2021/hash/9d86d83f925f2149e9edb0ac3b49229c-Abstract.html

Government Digital Service. (2025, February 10). *Artificial intelligence playbook for the UK Government*. GOV.UK. https://www.gov.uk/government/publications/ai-playbook-for-the-uk-government/artificial-intelligence-playbook-for-the-uk-government-html

Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? *Advances in Neural Information Processing Systems, 35*, 507-520. https://proceedings.neurips.cc/paper_files/paper/2022/hash/0378c7692da36807bdec87ab043cdadc-Abstract-Datasets_and_Benchmarks.html

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. In D. Precup & Y. W. Teh (Eds.), *Proceedings of the 34th International*

Conference on Machine Learning (Vol. 70, pp. 1321-1330). PMLR. https://proceedings.mlr.press/v70/guo17a.html

Hendrickx, K., Perini, L., Van der Plas, D., Meert, W., & Davis, J. (2024). Machine learning with a reject option: A survey. *Machine Learning, 113*(5), 3073-3110. https://doi.org/10.1007/s10994-024-06534-x

HM Treasury. (2017). *Guidance for publishing spend over £25,000*. GOV.UK. https://assets.publishing.service.gov.uk/media/5a821d93ed915d74e6235dd7/guidance_for_publishing_spend.pdf

Kapoor, S., Cantrell, E. M., Peng, K., Pham, T. H., Bail, C. A., Gundersen, O. E., Hofman, J. M., Hullman, J., Lones, M. A., Malik, M. M., Nanayakkara, P., Poldrack, R. A., Raji, I. D., Roberts, M., Salganik, M. J., Serra-Garcia, M., Stewart, B. M., Vandewiele, G., & Narayanan, A. (2024). REFORMS: Consensus-based recommendations for machine-learning-based science. *Science Advances, 10*(18), Article eadk3452. https://doi.org/10.1126/sciadv.adk3452

Kapoor, S., & Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns, 4*(9), Article 100804. https://doi.org/10.1016/j.patter.2023.100804

Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., & Gebru, T. (2019). Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency* (pp. 220-229). Association for Computing Machinery. https://doi.org/10.1145/3287560.3287596

Ovadia, Y., Fertig, E., Ren, J., Nado, Z., Sculley, D., Nowozin, S., Dillon, J. V., Lakshminarayanan, B., & Snoek, J. (2019). Can you trust your model's uncertainty?

Evaluating predictive uncertainty under dataset shift. *Advances in Neural Information Processing Systems, 32*. https://proceedings.neurips.cc/paper_files/paper/2019/hash/8558cb408c1d76621371888657d2eb1d-Abstract.html

Parasuraman, R., & Manzey, D. H. (2010). Complacency and bias in human use of automation: An attentional integration. *Human Factors, 52*(3), 381-410. https://doi.org/10.1177/0018720810376055

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825-2830. https://jmlr.org/papers/v12/pedregosa11a.html

scikit-learn developers. (2026a). *Common pitfalls and recommended practices* (Version 1.9.1) [Computer software documentation]. scikit-learn. Retrieved October 8, 2026, from https://scikit-learn.org/stable/common_pitfalls.html

scikit-learn developers. (2026b). *Feature extraction* (Version 1.9.1) [Computer software documentation]. scikit-learn. Retrieved October 10, 2026, from https://scikit-learn.org/stable/modules/feature_extraction.html

scikit-learn developers. (2026c). *Linear models* (Version 1.9.1) [Computer software documentation]. scikit-learn. Retrieved October 10, 2026, from https://scikit-learn.org/stable/modules/linear_model.html

scikit-learn developers. (2026d). *Metrics and scoring: Quantifying the quality of predictions* (Version 1.9.1) [Computer software documentation]. scikit-learn. Retrieved October 8, 2026, from https://scikit-learn.org/stable/modules/model_evaluation.html

scikit-learn developers. (2026e). *Preprocessing data* (Version 1.9.1) [Computer software documentation]. scikit-learn. Retrieved October 10, 2026, from https://scikit-learn.org/stable/modules/preprocessing.html

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems, 28*. https://proceedings.neurips.cc/paper_files/paper/2015/hash/86df7dcfd896fcaf2674f757a2463eba-Abstract.html

van den Goorbergh, R., van Smeden, M., Timmerman, D., & Van Calster, B. (2022). The harm of class imbalance corrections for risk prediction models: Illustration and simulation using logistic regression. *Journal of the American Medical Informatics Association, 29*(9), 1525-1534. https://doi.org/10.1093/jamia/ocac093